МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Школа разработки видеоигр

Практическое занятие №1

«Каркас игрового движка и базовая интеграция рендера»

по дисциплине «Архитектура игровых движков»

Выполнили:
студенты группы J3300
Власов Д. Н.
Шабанов Т. О.
Проверил:
доцент практики
Павлович А. А.

г. Санкт-Петербург
2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. Цель работы 3
2. Задачи работы 3
ХОД РАБОТЫ 4
1. Описание и архитектура 4
2. Технологический стек 4
3. Система сборки 5
4. Точка входа 6
5. Основной класс Application 6
6. Система окон 7
7. Система состояний игры 8
8. Расчёт времени 9
9. Система логирования 9
10. Адаптер рендеринга 9
11. Главный цикл и интеграция 10
12. Работа игрового движка 10
ЗАКЛЮЧЕНИЕ 12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 13
ПРИЛОЖЕНИЕ 1 14
ПРИЛОЖЕНИЕ 2 15
ПРИЛОЖЕНИЕ 3 16

ВВЕДЕНИЕ

1. Цель работы

Целью данного практического занятия является создание минимального рабочего каркаса игрового движка с поддержкой игрового цикла и визуализации графического вывода через адаптер рендеринга.

2. Задачи работы

Задачами данного практического занятия являются:

1) Реализация класса Application, управляющего жизненным циклом приложения, включая инициализацию, запуск основного цикла и завершение программы.

2) Разработка структуры для хранения и смены различных игровых режимов (например, меню, игровое состояние, паузы), используя паттерн «Состояние».

3) Организация точной обработки временного интервала между кадрами для обеспечения плавности изменений в игре (анимации, физики и т. д.).

4) Настройка механизма ведения журнала для записи важных событий и ошибок.

5) Создание интерфейса адаптера рендеринга, позволяющего подключать различные графические API (OpenGL, DirectX, Vulkan и др.) без внесения изменений в основной код.

6) Реализация отображения простейшего геометрического элемента (треугольника или квадрата) в графическом окне через созданный адаптер.

ХОД РАБОТЫ

1. Описание и архитектура

NeneEngine — это минимальный каркас игрового движка, разработанный на C++20 в рамках практического занятия №1.

Ссылка на репозиторий с исходным кодом игрового движка NeneEngine на платформе хостинга репозиториев GitHub представлена в Приложении 1.

Архитектура игрового движка NeneEngine построена по модульному принципу с разделением ответственности и соблюдением принципов SOLID. Основная задумка – создать минимальный, но расширяемый каркас игрового движка, в котором все ключевые подсистемы взаимодействуют через абстрактные интерфейсы и паттерны проектирования. Такой подход позволит в любой момент заменить реализацию любой части движка без единой правки в основном коде приложения. Центральным элементом выступает класс Application, который выступает оркестратором всего жизненного цикла.

2. Технологический стек

В процессе реализации игрового движка NeneEngine был использован следующий технологический стек:

1) C++ 20 – основной язык программирования, обеспечивающий высокую производительность, современные возможности ООП и контроль памяти.

2) CMake – кроссплатформенная система сборки, отвечающая за управление зависимостями и генерацию проектов.

3) vcpkg – менеджер пакетов Microsoft, используемый для автоматической установки и интеграции всех сторонних библиотек.

4) Git и GitHub – распределённая система контроля версий и платформа хостинга репозитория с полной историей разработки.

5) EASTL – альтернатива стандартной библиотеке шаблонов STL от Electronic Arts, оптимизированная для игровых движков.

6) spdlog – библиотека асинхронного логирования с поддержкой уровней

логирования и другими настройками.

7) Diligent Graphics – мультиплатформенный низкоуровневый графический фреймворк.

8) Win32 API – нативный API Windows для создания и управления оконной подсистемой.

9) nlohmann/json – библиотека для работы с JSON-форматом конфигурационных файлов.

10) EnTT – библиотека Entity Component System, предназначенная для компонентной архитектуры игровых объектов.

3. Система сборки

Игровой движок NeneEngine использует двухуровневую систему сборки на базе CMake.

Корневой CMake-файл CMakeLists.txt задаёт глобальную конфигурацию: минимальную версию CMake, стандарт C++, проектные метаданные и специальную настройку Diligent Engine (отключение всех графических backend-ов кроме DirectX 12). Он подключает внешний модуль DiligentCore и директорию src, а также применяет исправления предупреждений компилятора для вспомогательных целей.

Полный листинг корневого CMake-файла CMakeLists.txt продемонстрирован в Приложении 2.

CMake-файл CMakeLists.txt в папке src выполняет рекурсивный сбор исходников, исключает тестовые файлы, создаёт Windows-приложение NeneEngine, настраивает include-директории, находит пакеты через find_package (vcpkg) и производит линковку всех библиотек. Автоматически копируются требуемые DLL Diligent.

Полный листинг CMake-файла CMakeLists.txt в папке src продемонстрирован в Приложении 3.

4. Точка входа

Файл main.cpp содержит Windows-специфическую точку входа wWinMain вместо стандартного main. В try-блоке создаётся экземпляр Application, вызывается метод Init для инициализации окна и рендерера. При успешной инициализации запускается главный цикл через Run. В случае ошибки инициализации или исключения выводится сообщение в cerr и программа завершается с кодом ошибки.

Листинг файла с точкой входа:

```
int WINAPI wWinMain(HINSTANCE, HINSTANCE, LPWSTR, int) {
    try {
        NeneEngine::Application app;
        if (!app.Init(1280, 720, "NeneEngine")) {
            std::cerr << "Failed to initialize application\n";
            return -1;
        }
        app.Run();
        return 0;
    }
    catch (const std::exception& e) {
        std::cerr << "Fatal error: " << e.what() << "\n";
        return -1;
    }
}
```

5. Основной класс Application

Класс Application является центральным оркестратором жизненного цикла приложения. Он управляет инициализацией, запуском главного цикла и корректным завершением работы. Класс владеет уникальными указателями на абстрактные интерфейсы окна и рендерера, экземпляром таймера и атомарными флагами состояния, обеспечивающими потокобезопасное управление запуском и паузой.

В процессе инициализации происходит создание оконного объекта и адаптера рендеринга с передачей параметров разрешения и заголовка. Далее главный цикл последовательно обрабатывает сообщения, обновляет состояния и выполняет рендеринг, а метод завершения гарантирует освобождение всех ресурсов.

Ключевой листинг класса Application:

```
// Application.h
class Application {
public:
    bool Init(uint32_t width = 1280, uint32_t height = 720, const
std::string& title = "NeneEngine");
    void Run();
    void RequestShutdown();
private:
    eastl::unique_ptr<WindowInterface> m_window;
    eastl::unique_ptr<RenderAdapter> m_renderer;
    GameTimer m_timer;
    std::atomic<bool> m_running{ false };
    std::atomic<bool> m_isPaused{ false };
};
// Application.cpp (фрагмент Run())
void Application::Run() {
    m_running = true;
    m_timer.Reset();
    while (m_running && !m_window->ShouldClose()) {
        m_window->PumpMessages();
        m_timer.Tick();
        if (!m_isPaused) {
            m_renderer->BeginFrame();
            m_renderer->EndFrame();
            m_renderer->Present();
            CalculateFrameStats();
        } else {
            Sleep(100);
        }
    }
}
```

6. Система окон

Абстрактный интерфейс WindowInterface отделяет платформенно-зависимый код от остальной логики движка посредством виртуальных методов создания, уничтожения, обработки сообщения и получения дескрипторов. Конкретная реализация Win32Window наследует интерфейс и использует нативный Win32 API.

Win32Window создаёт окно через CreateWindowEx, регистрирует оконную процедуру WndProc, обрабатывает очередь сообщений в PumpMessages и освобождает ресурсы в деструкторе.

Ключевой листинг классов WindowInterface и Win32Window:

```
// WindowInterface.h
class WindowInterface {
public:
    virtual bool Create(uint32_t width, uint32_t height, const
```

```
std::string& title) = 0;
    virtual void PumpMessages() = 0;
    virtual HWND GetHWND() const = 0;
    // ...
};
// Win32Window.cpp (фрагмент Create и PumpMessages)
bool  Win32Window::Create(uint32_t  width,  uint32_t  height,  const
std::string& title) {
    // регистрация класса окна, CreateWindowEx, ShowWindow
}
void Win32Window::PumpMessages() {
    while (PeekMessage(&msg, nullptr, 0, 0, PM_REMOVE)) {
        if (msg.message == WM_QUIT) m_shouldClose = true;
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
}
```

7. Система состояний игры

Абстрактный класс GameStateInterface определяет контракт для всех игровых режимов, включая обязательные методы входа, выхода, обновления с учётом дельта-времени, обработки ввода, а также виртуальные флаги прозрачности и паузы. GameStateMachine реализует стек состояний на основе вектора уникальных указателей, обеспечивая операции смены, push/pop состояний, делегирование обновления и обработки ввода текущему верхнему состоянию.

Ключевой листинг класса GameStateMachine:

```
void  GameStateMachine::PushState(eastl::unique_ptr<GameStateInterface>
newState) {
    if (!m_states.empty()) m_states.back()->OnExit();
    m_states.push_back(eastl::move(newState));
    m_states.back()->OnEnter();
}
void GameStateMachine::Update(float deltaTime) {
    for (int i = static_cast<int>(m_states.size()) - 1; i >= 0; --i) {
        m_states[i]->Update(deltaTime);
        if (!m_states[i]->IsPausing()) break;
    }
}
```

8. Расчёт времени

Класс GameTimer построен на базе std::chrono::steady_clock и хранит базовое время, накопленную паузу, предыдущее и текущее временные метки.

Ключевой листинг GameTimer:

```
void GameTimer::Tick() {
    if (mStopped) { mDeltaTime = 0.0; return; }
    auto currTime = clock::now();
    mCurrTime = currTime;
    mDeltaTime = std::chrono::duration<double>(currTime - mPrevTime).count();
    mPrevTime = currTime;
}
float GameTimer::DeltaTime() const { return static_cast<float>(mDeltaTime); }
```

9. Система логирования

CustomLogger представляет собой потокобезопасный синглетон-обёртку над библиотекой spdlog. Он инициализирует асинхронный или синхронный логгер с поддержкой цветного вывода в консоль и записи в файл, устанавливает уровень логирования и представляет шаблонные методы для уровней trace, debug, info, warn, error и critical.

Ключевой листинг CustomLogger:

```
bool CustomLogger::Initialize(const std::string& logFileName, bool async, ...) {
    std::vector<spdlog::sink_ptr> sinks;
    auto console = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
    // добавление file sink и async_logger
    m_logger = async ? std::make_shared<spdlog::async_logger>(...) : ...;
    m_logger->set_level(logLevel);
    spdlog::set_default_logger(m_logger);
}
```

10. Адаптер рендеринга

Абстрактный интерфейс RenderAdapter определяет единый контракт для графического бэкенда: инициализацию с HWND и разрешением, завершение работы, управление кадром, вывод на экран и обработку ресайза. Реализация DiligentDX12Adapter использует Diligent Graphics Engine (DirectX 12 backend) и создаёт рендеринг-пайплайн с встроенными HLSL-шейдерами для отрисовки

цветного треугольника.

Ключевой листинг DiligentDX12Adapter:

```
bool DiligentDX12Adapter::Init(HWND hwnd, uint32_t width, uint32_t height) {
    // CreateDeviceAndContextsD3D12 + CreateSwapChainD3D12
    CreateResources(); // настройка PSO с vertex/pixel shader
}
void DiligentDX12Adapter::BeginFrame() {
    m_pImmediateContext->ClearRenderTarget(...);
}
```

11. Главный цикл и интеграция

В главном цикле класса Application последовательно выполняются тик таймера, обработка оконных сообщений, делегирование ввода и обновления текущему состоянию через GameStateMachine, начало и окончание кадра рендерера с выводом на экран и расчётом статистики кадров. Проект использует EASTL для всех контейнеров и умных указателей в целях максимальной производительности и контроля памяти, а все внешние зависимости подключаются через vcpkg и CMake.

Ключевой листинг главного цикла:

```
while (m_running && !m_window->ShouldClose()) {
    m_window->PumpMessages();
    m_timer.Tick();
    if (!m_isPaused) {
        m_renderer->BeginFrame();
        m_renderer->EndFrame();
        m_renderer->Present();
        CalculateFrameStats();
    }
}
```

12. Работа игрового движка

В демонстрационном режиме приложение открывает окно 1280x720, в котором через Diligent Graphics (DirectX 12) отрисовывается цветной треугольник с HLSL-шейдерами. В заголовке окна в реальном времени выводится статистика FPS и времени кадра. Работают таймер, система

логирования и базовая система состояний. При закрытии окна цикл завершается, все ресурсы освобождаются корректно.

Демонстрация работы игрового движка представлена на рисунке 1.

Рис. 1 Отрисовка тестового геометрического примитива в демонстрационном окне

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного практического занятия был разработан рабочий каркас игрового движка на языке программирования C++ с поддержкой полного игрового цикла и базовой визуализацией. Реализованы классы для управления состояниями, расчёта времени, логирования и поддержки различных графических API. Через адаптер рендеринга отображён тестовый геометрический примитив.

Прототип игрового движка полностью соответствует поставленным задачам, демонстрирует работоспособность архитектуры и закладывает фундамент для дальнейшего расширения движка.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Джейсон Г. Игровой движок. – СПб.: Издательский дом Питер, 2021 г. – 1136 с.

2. Нильстрём Р. Паттерны программирования игр. – М.: Эксмо, 2021 г. – 432 с.

3. Купер Дж. Основы архитектуры игровых движков. – М.: Наука, 2018 г. – 2022 с.

4. https://github.com/electronicarts/EASTL/tree/master/doc – документация альтернативной стандартной библиотеки шаблонов EASTL.

5. https://spdlog.readthedocs.io/en/latest/ – документация библиотеки логирования spdlog.

6. https://diligentgraphics.github.io/docs/ – документация графического фреймворка Diligent Graphics.

7. https://learn.microsoft.com/en-us/windows/win32/api/ – документация API оконной системы Windows Win32API.

8. https://nlohmann.github.io/json/ – документация библиотеки для работы с JSON-форматом nlohmann/json.

9. https://skypjack.github.io/entt/ – документация библиотеки Entity Component System EnTT.

ПРИЛОЖЕНИЕ 1

Репозиторий игрового движка NeneEngine на платформе GitHub

https://github.com/BennySpace/NeneEngine

ПРИЛОЖЕНИЕ 2

Листинг корневого CMake-файла CMakeLists.txt

```
# NeneEngine/CMakeList.txt : Top-level CMake project file, do global
configuration
cmake_minimum_required (VERSION 3.12)
project (
            NeneEngine
            VERSION 0.1
            DESCRIPTION "Game engine"
            LANGUAGES CXX
)
set(CMAKE_CXX_STANDARD          20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS        OFF)
# Diligent Engine configs
set(DILIGENT_BUILD_SAMPLES OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TOOLS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_FX      OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TESTS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_NO_DIRECT3D11 ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_VULKAN     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_OPENGL     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_GLES       ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_METAL      ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_DIRECT3D12 OFF CACHE BOOL "" FORCE)
add_subdirectory ("external/DiligentEngine/DiligentCore")
foreach(target_name SPIRV-Tools-opt SPIRV-Tools SPIRV-Tools-core)
    if(TARGET ${target_name})
        get_target_property(aliased_target ${target_name}
ALIASED_TARGET)
        if(aliased_target)
            target_compile_options(${aliased_target} PRIVATE /wd5232
/wd4717)
            set_target_properties(${aliased_target} PROPERTIES
COMPILE_WARNING_AS_ERROR OFF)
            message(STATUS "[Diligent Fix] Applied to aliased target:
${aliased_target}")
        else()
            target_compile_options(${target_name} PRIVATE /wd5232
/wd4717)
            set_target_properties(${target_name} PROPERTIES
COMPILE_WARNING_AS_ERROR OFF)
            message(STATUS "[Diligent Fix] Applied to target:
${target_name}")
        endif()
    endif()
endforeach()
add_subdirectory ("src")
# add_subdirectory(tests)
```

ПРИЛОЖЕНИЕ 3

Листинг CMake-файла CMakeLists.txt для CPP-файлов

```
# NeneEngine/src/CMakeList.txt : CMake project for NeneEngine, include
source and define project specific logic here.
file(GLOB_RECURSE SOURCES CONFIGURE_DEPENDS
    "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/include/*.h"
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp"
)
list(FILTER SOURCES EXCLUDE REGEX
    "(.*_test\\.cpp$|.*Test\\.cpp$|.*\\.t\\.cpp$|.*main_test\\.cpp$)"
)
add_executable(NeneEngine WIN32 ${SOURCES}
"${CMAKE_SOURCE_DIR}/main.cpp")
target_include_directories(NeneEngine
            PUBLIC

            $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../include>

            $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../external>
)
find_package(spdlog                             CONFIG REQUIRED)
find_package(nlohmann_json          CONFIG REQUIRED)
find_package(EASTL                              CONFIG REQUIRED)
find_package(EnTT                               CONFIG REQUIRED)
target_link_libraries(NeneEngine
            PRIVATE
                        spdlog::spdlog
                        nlohmann_json::nlohmann_json
                        EASTL
                        EnTT::EnTT
                        Diligent-GraphicsEngineD3D12-shared
)
if(COMMAND copy_required_dlls)
    copy_required_dlls(NeneEngine)
else()
    message(WARNING "copy_required_dlls не найдена — возможно,
Diligent не подгрузился")
endif()
set_property(DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR} PROPERTY
VS_STARTUP_PROJECT NeneEngine)
target_compile_features(NeneEngine PUBLIC cxx_std_20)
```